

git

Test de repaso — parte 1: trabajando en local

100 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al examen. TODAS las preguntas del examen saldrán de este banco de preguntas.

Bloque 1

Conceptos de SCV y por qué git

Pregunta 1

¿Qué es un Sistema de Control de Versiones (SCV)?

- A) Una aplicación que gestiona y guarda versiones de los cambios de un proyecto
- B) Un editor de texto para escribir código
- C) Un servidor donde se aloja una página web
- D) Un lenguaje de programación

Pregunta 2

¿Cuál de las siguientes NO es una ventaja de usar un SCV?

- A) Permite volver a estados previos del proyecto
- B) Aumenta automáticamente el rendimiento del código
- C) Registra quién y cuándo hizo cada cambio
- D) Facilita la colaboración entre varios usuarios

Pregunta 3

¿En qué año se creó git?

- A) 1998
- B) 2010
- C) 2005
- D) 2015

Pregunta 4

¿Qué característica define la arquitectura de git?

- A) Centralizada
- B) Basada en la nube exclusivamente
- C) Cliente-servidor obligatorio
- D) Distribuida

Pregunta 5

¿Cuál de estos es un Sistema de Control de Versiones distinto a git?

- A) Subversion
- B) Docker
- C) Vim
- D) Node.js

Pregunta 6

¿Qué es Git LFS?

- A) Un SCV alternativo a git
- B) Un plugin de git para gestionar ficheros binarios grandes
- C) Un servidor remoto de git
- D) Un editor de texto integrado en git

Pregunta 7

¿Cuál de estos SCV es especialmente usado en el desarrollo de videojuegos?

- A) Subversion
- B) Mercurial
- C) Perforce
- D) Git LFS

Pregunta 8

¿Qué significa que git sea "de código abierto"?

- A) Que solo lo puede usar quien pague licencia
- B) Que no tiene versión de escritorio
- C) Que solo funciona en Linux
- D) Que su código fuente es público y se puede consultar/modificar

Pregunta 9

Sin usar un SCV, ¿cuál de estos problemas es más probable que ocurra en un proyecto?

- A) Perder el histórico de cambios y tener ficheros tipo "final_v2_definitivo.zip"
- B) El código se ejecuta más rápido
- C) El proyecto ocupa menos espacio en disco
- D) Los ficheros se compilan automáticamente

Pregunta 10

¿Cuál de las siguientes afirmaciones sobre git es correcta?

- A) Git solo permite trabajar en remoto, nunca en local
- B) Git permite trabajar completamente en local, sin ningún servidor remoto
- C) Git necesita obligatoriamente una cuenta de GitHub para funcionar
- D) Git solo sirve para ficheros de texto plano

Bloque 2

git config

Pregunta 11

¿Para qué sirve principalmente `git config` ?

- A) Para crear un nuevo repositorio
- B) Para ver el historial de commits
- C) Para establecer opciones de git, como tu nombre y email
- D) Para borrar un repositorio

Pregunta 12

¿Qué comando configura tu nombre de forma global (para todos tus repositorios)?

- A) `git config user.name "nombre"`
- B) `git init user.name "nombre"`
- C) `git set user.name "nombre"`
- D) `git config --global user.name "nombre"`

Pregunta 13

¿Qué diferencia hay entre `git config user.name` y `git config --global user.name` ?

- A) El primero afecta solo al repositorio actual, el segundo a todos los repositorios
- B) Ninguna, son idénticos
- C) El primero es solo para Windows
- D) El segundo borra la configuración anterior

Pregunta 14

¿Por qué es importante configurar `user.email` antes de hacer commits?

- A) Porque si no, git no arranca
- B) Porque git necesita saber quién eres para registrar adecuadamente tus cambios
- C) Porque se usa para enviar la contraseña de git
- D) Porque es obligatorio para instalar git

Pregunta 15

¿Qué comando usarías para ver toda tu configuración global de git?

- A) `git status --global`
- B) `git init --list`
- C) `git config --global --list`
- D) `git show --global`

Pregunta 16

Si `git config user.name` no devuelve nada, ¿qué significa?

- A) Que git está roto y hay que reinstalarlo
- B) Que el repositorio no existe
- C) Que no hay conexión a internet
- D) Que el nombre de usuario no está configurado

Pregunta 17

¿Cuál de estas opciones se puede configurar con `git config` ?

- A) Nombre de usuario, email y otras preferencias de comportamiento de git
- B) Solo el nombre de usuario
- C) Solo el email
- D) Solo el tema de colores de la terminal

Pregunta 18

Un alumno configura su nombre en local (sin `--global`) dentro de `proyecto-A`. Si luego crea `proyecto-B` con `git init`, ¿tendrá ese nombre configurado en `proyecto-B`?

- A) Sí, siempre
- B) No, porque la configuración local solo aplica al repositorio donde se ejecutó
- C) Solo si `proyecto-B` está dentro de `proyecto-A`
- D) Solo los lunes

Bloque 3

`git init` y la carpeta `.git`

Pregunta 19

¿Qué hace el comando `git init` ?

- A) Sube el proyecto a GitHub
- B) Elimina el repositorio actual
- C) Crea un nuevo repositorio de git en el directorio actual
- D) Configura el nombre de usuario

Pregunta 20

Tras ejecutar `git init`, ¿qué carpeta oculta se crea?

- A) `.repo`
- B) `.gitconfig`
- C) `.svn`
- D) `.git`

Pregunta 21

Si ejecutas `git init` en una carpeta que ya tiene archivos, ¿qué ocurre con esos archivos?

- A) No se modifican, simplemente el directorio empieza a poder ser rastreado
- B) Se borran
- C) Se modifican automáticamente
- D) Se mueven a otra carpeta

Pregunta 22

¿Dónde guarda git toda la información del historial de un repositorio (commits, ramas, etc.)?

- A) En un servidor remoto por defecto
- B) En la carpeta oculta `.git`
- C) En la memoria RAM, se pierde al reiniciar
- D) En un fichero llamado `historial.txt`

Pregunta 23

¿Es necesario tener conexión a internet para ejecutar `git init` y hacer commits?

- A) Sí, siempre
- B) Solo la primera vez
- C) No, git funciona completamente en local
- D) Solo si usas Windows

Pregunta 24

Si borras manualmente la carpeta `.git` de un proyecto, ¿qué sucede?

- A) Nada, es una carpeta decorativa
- B) Se sube automáticamente a GitHub como backup
- C) Git la vuelve a crear sola
- D) Se pierde todo el historial de versiones de ese repositorio

Pregunta 25

¿Cuál de estas afirmaciones sobre `git init` es correcta?

- A) Se puede ejecutar en cualquier directorio para empezar a versionarlo
- B) Solo se puede ejecutar una vez en todo el ordenador
- C) Requiere permisos de administrador siempre
- D) Borra el contenido previo del directorio

Pregunta 26

Después de `git init`, ¿el proyecto ya tiene commits en su historial?

- A) Sí, automáticamente se crea un commit inicial vacío
- B) No, el historial está vacío hasta que se haga el primer commit
- C) Sí, uno por cada fichero existente
- D) Depende de la versión de git

Bloque 4

Los 3 estados / áreas de git

Pregunta 27

¿Cuáles son las 3 áreas por las que pasa un cambio en git?

- A) Editor, terminal, navegador
- B) Local, remoto, nube
- C) Working directory, staging area, repository
- D) Untracked, deleted, archived

Pregunta 28

Un fichero que git no ha visto nunca (recién creado) está en estado...

- A) Staged
- B) Modified
- C) Committed
- D) Untracked

Pregunta 29

Un fichero que ya está siendo rastreado por git y tiene cambios sin preparar está en estado...

- A) Modified
- B) Untracked
- C) Staged
- D) Ignored

Pregunta 30

Un fichero que está listo para el próximo commit está en estado...

- A) Untracked
- B) Staged
- C) Modified
- D) Deleted

Pregunta 31

¿Qué comando mueve un fichero del working directory a la staging area?

- A) `git commit`
- B) `git status`
- C) `git add`
- D) `git init`

Pregunta 32

¿Qué comando mueve los cambios de la staging area al repository (historial)?

- A) `git add`
- B) `git status`
- C) `git config`
- D) `git commit`

Pregunta 33

Tras hacer `git commit`, ¿el working directory queda "congelado" y ya no se puede volver a editar?

- A) No, el flujo es un ciclo: puedes seguir editando y volver a empezar
- B) Sí, hay que crear un repositorio nuevo para seguir
- C) Sí, solo se puede editar en la staging area
- D) No, pero solo se puede editar una vez más

Pregunta 34

Si modificas un fichero que ya habías confirmado en un commit anterior, ¿en qué estado pasa a estar?

- A) Sigue en staged
- B) Pasa a modified (tiene cambios sin preparar)
- C) Pasa a untracked
- D) Desaparece del repositorio

Bloque 5

```
git status
```

Pregunta 35

¿Qué información proporciona `git status` ?

- A) El historial completo de commits
- B) La configuración global de git
- C) El estado actual de los ficheros del repositorio
- D) El contenido de un fichero concreto

Pregunta 36

¿ `git status` modifica los archivos o el repositorio?

- A) Sí, siempre
- B) Solo si se usa con `-m`
- C) Solo los ficheros untracked
- D) No, solo informa, no modifica nada

Pregunta 37

En la salida de `git status`, ¿en qué sección aparecen los ficheros nuevos que nunca se han añadido?

- A) Untracked files
- B) Changes to be committed
- C) Changes not staged for commit
- D) Committed files

Pregunta 38

En la salida de `git status`, ¿en qué sección aparece un fichero que ya se ha preparado con `git add` pero no confirmado?

- A) Untracked files
- B) Changes to be committed
- C) Deleted files
- D) Ignored files

Pregunta 39

Si ejecutas `git status` justo después de `git init` en una carpeta vacía, ¿qué verás?

- A) Un error
- B) Una lista de commits
- C) Un mensaje indicando que no hay commits y no hay cambios que mostrar
- D) La configuración global

Pregunta 40

Tras hacer `git add archivo.txt` y confirmarlo con `git commit`, ¿qué mostrará `git status` si no has vuelto a tocar nada?

- A) `archivo.txt` como untracked
- B) `archivo.txt` como staged
- C) Un error porque ya se ha confirmado
- D) Que no hay cambios pendientes (working tree clean)

Pregunta 41

¿Cuál de estas frases sobre `git status` es correcta?

- A) Es un comando informativo que se puede ejecutar en cualquier momento sin riesgo
- B) Es el comando que se usa para deshacer un commit
- C) Borra los ficheros untracked automáticamente
- D) Sube los cambios a GitHub

Pregunta 42

Un fichero aparece a la vez en "Changes to be committed" y en "Changes not staged for commit". ¿Qué significa esto?

- A) Es un error de git
- B) Hay una versión del fichero ya preparada (staged) y, además, cambios posteriores sin preparar
- C) El fichero se va a borrar en el próximo commit
- D) El fichero está en `.gitignore`

Bloque 6

```
git add
```

Pregunta 43

¿Qué hace `git add archivo.txt` ?

- A) Crea el fichero `archivo.txt`
- B) Confirma los cambios de `archivo.txt` en el historial
- C) Prepara `archivo.txt` en la staging area para el próximo commit
- D) Borra `archivo.txt`

Pregunta 44

¿Qué hace `git add .` ?

- A) Añade solo el fichero llamado "."
- B) Elimina todos los cambios
- C) Muestra el estado del repositorio
- D) Añade todos los cambios del directorio actual y subdirectorios

Pregunta 45

¿ `git add` guarda los cambios de forma permanente en el historial del proyecto?

- A) No, solo los prepara; hace falta `git commit` para guardarlos en el historial
- B) Sí, es equivalente a un commit
- C) Sí, pero solo si se usa `-m`
- D) No, `git add` no hace nada

Pregunta 46

¿Se puede usar `git add` tanto para ficheros nuevos como para ficheros ya rastreados con cambios?

- A) No, solo para ficheros nuevos
- B) Sí, sirve para ambos casos
- C) No, solo para ficheros ya rastreados
- D) Solo si el fichero está en `.gitignore`

Pregunta 47

¿Qué riesgo tiene usar `git add .` sin un `.gitignore` configurado?

- A) Ninguno, es totalmente seguro siempre
- B) Borra los ficheros ignorados
- C) Puede añadir ficheros que no querías versionar (logs, credenciales, dependencias...)
- D) Elimina la carpeta `.git`

Pregunta 48

Después de `git add archivo.txt`, ¿en qué área se encuentra el fichero?

- A) Working directory
- B) Repository
- C) Untracked
- D) Staging area

Pregunta 49

Si modificas `archivo.txt` después de haberlo añadido con `git add` pero antes de hacer `git commit`, ¿qué ocurre?

- A) La nueva modificación aparece como "no preparada", habría que volver a hacer `git add`
- B) El commit incluirá automáticamente los nuevos cambios
- C) Da un error y no deja continuar
- D) Se deshace el `git add` anterior

Pregunta 50

¿Cuál de estos comandos prepara únicamente el fichero `index.html` ?

- A) `git add .`
- B) `git add index.html`
- C) `git commit index.html`
- D) `git status index.html`

Pregunta 51

Un alumno ejecuta `git add` sin ningún argumento. ¿Qué ocurre?

- A) Añade todos los ficheros automáticamente, igual que `git add .`
- B) Se confirma un commit vacío
- C) Git muestra un error/aviso pidiendo especificar qué añadir (no añade nada por sí solo)
- D) Se borra el repositorio

Pregunta 52

¿Qué comando comprobarías justo después de `git add` para verificar que el fichero ha pasado correctamente a la staging area?

- A) `git log`
- B) `git init`
- C) `git config --list`
- D) `git status`

Bloque 7

```
git commit
```

Pregunta 53

¿Qué hace `git commit -m "mensaje"` ?

- A) Guarda de forma permanente en el historial los cambios que estaban en la staging area
- B) Prepara los cambios en la staging area
- C) Borra el historial anterior
- D) Sube los cambios a un servidor remoto

Pregunta 54

¿Qué ocurre si ejecutas `git commit` sin la opción `-m` ?

- A) Da un error inmediato y no continúa
- B) Se abre un editor de texto (normalmente Vim) para escribir el mensaje
- C) Se usa automáticamente el mensaje del commit anterior
- D) El commit se hace sin mensaje

Pregunta 55

Si te quedas "atrapado" en Vim tras un `git commit` sin `-m`, ¿cómo sales guardando el mensaje?

- A) Cerrando la terminal
- B) `Ctrl+C`
- C) `Esc` y luego `:wq` + `Enter`
- D) `Alt+F4`

Pregunta 56

¿Qué ficheros incluye un `git commit` ?

- A) Todos los ficheros del proyecto, estén o no en staging
- B) Únicamente los ficheros untracked
- C) Ninguno, hasta que no se haga `git push`
- D) Únicamente los ficheros que están en la staging area

Pregunta 57

¿Por qué se dice que un commit crea un "snapshot" del proyecto?

- A) Porque guarda una instantánea del estado de los ficheros preparados en ese momento
- B) Porque hace una foto literal de la pantalla
- C) Porque comprime el proyecto en un zip
- D) Porque genera un vídeo del proceso

Pregunta 58

Según las buenas prácticas vistas en clase, ¿cómo debería redactarse un mensaje de commit?

- A) En pasado y muy largo, contando toda la sesión de trabajo
- B) En imperativo y describiendo el cambio de forma concisa
- C) Solo con el nombre del autor
- D) Vacío, para que sea más rápido

Pregunta 59

¿Qué recomienda la buena práctica de "commits atómicos"?

- A) Hacer un único commit gigante al final del proyecto
- B) Hacer un commit por cada línea de código
- C) Que cada commit represente un único cambio lógico
- D) No usar mensajes de commit

Pregunta 60

¿Debería incluirse una contraseña o clave API en un commit?

- A) Sí, así queda documentado
- B) Solo si el repositorio es local
- C) Solo en el primer commit
- D) No, nunca; para evitarlo conviene usar `.gitignore`

Pregunta 61

Tras hacer `git commit -m "Primer commit"`, ¿puede ejecutarse otro `git commit` inmediatamente sin haber tocado nada?

- A) No tiene sentido: no hay nada en la staging area, así que no habría cambios que confirmar
- B) Sí, y crea un commit idéntico
- C) Sí, obligatoriamente cada 5 minutos
- D) No, git lo bloquea permanentemente

Pregunta 62

¿Qué comando usarías para comprobar que un commit se ha registrado correctamente?

- A) `git config`
- B) `git log`
- C) `git init`
- D) `git add`

Bloque 8

```
git log y git log --oneline
```


Pregunta 63

¿Qué muestra `git log` ?

- A) La configuración de git
- B) Solo los ficheros untracked
- C) El historial de commits del repositorio
- D) El contenido de `.gitignore`

Pregunta 64

¿Cuál de estos datos NO aparece en la salida estándar de `git log` ?

- A) El hash del commit
- B) El autor
- C) La fecha
- D) La contraseña del autor

Pregunta 65

¿Qué es el hash de un commit (SHA-1)?

- A) Un identificador único del commit
- B) El nombre del autor
- C) La fecha de creación en otro formato
- D) El número de ficheros modificados

Pregunta 66

¿Qué ventaja tiene `git log --oneline` frente a `git log` ?

- A) Muestra más información
- B) Muestra un historial más compacto, una línea por commit
- C) Solo muestra el último commit
- D) Ordena los commits alfabéticamente

Pregunta 67

En `git log --oneline`, ¿qué se muestra de cada commit?

- A) Solo la fecha
- B) Solo el autor
- C) Hash corto y mensaje del commit
- D) El contenido completo del diff

Pregunta 68

Si un repositorio no tiene ningún commit todavía, ¿qué ocurre al ejecutar `git log` ?

- A) Da un error fatal e irrecoverable
- B) Crea un commit automáticamente
- C) Muestra el `git status`
- D) Muestra un historial vacío o un mensaje indicando que no hay commits

Pregunta 69

¿En qué orden muestra `git log` los commits por defecto?

- A) Del más reciente al más antiguo
- B) Alfabético por mensaje
- C) Del más antiguo al más reciente
- D) Aleatorio

Pregunta 70

¿ `git log` modifica el historial del repositorio?

- A) Sí, reordena los commits
- B) No, solo lo consulta
- C) Sí, borra los commits más antiguos
- D) Solo si se usa `--oneline`

Pregunta 71

Un compañero te pasa el hash corto `124c35f` de un commit visto con `git log --oneline`. ¿De qué commit completo es un fragmento?

- A) Del mensaje del commit
- B) Del nombre del autor codificado
- C) Del SHA-1 completo de ese commit
- D) De la fecha en hexadecimal

Pregunta 72

¿Qué comando ejecutarías para revisar rápidamente cuántos commits llevas y con qué mensajes, sin entrar en detalle de fechas y autores?

- A) `git status`
- B) `git config --list`
- C) `git add .`
- D) `git log --oneline`

Bloque 9

.gitignore

Pregunta 73

¿Para qué sirve el fichero `.gitignore` ?

- A) Para indicar a git qué ficheros o carpetas debe ignorar
- B) Para configurar el nombre de usuario
- C) Para guardar el historial de commits
- D) Para crear un nuevo repositorio

Pregunta 74

Un fichero listado en `.gitignore`, ¿aparece en `git status` como untracked?

- A) Sí, siempre
- B) No, git lo ignora y no lo muestra
- C) Solo si se usa `git add .`
- D) Solo la primera vez

Pregunta 75

¿Qué tipo de ficheros es buena práctica incluir en `.gitignore` ?

- A) El código fuente principal del proyecto
- B) El propio `README.md`
- C) Ficheros de log, dependencias y credenciales
- D) Los mensajes de commit

Pregunta 76

¿Cómo se ignorarían todos los ficheros con extensión `.log` en `.gitignore`?

- A) `log.*`
- B) `ignore(.log)`
- C) `-.log`
- D) `*.log`

Pregunta 77

¿Qué diferencia hay entre escribir `node_modules` y `node_modules/` en un `.gitignore`?

- A) Ninguna diferencia práctica relevante para este caso: ambas ignoran esa carpeta
- B) La primera ignora un fichero llamado exactamente `node_modules`, nunca una carpeta
- C) La segunda solo funciona en Windows
- D) La primera solo funciona dentro de subcarpetas

Pregunta 78

Un fichero ya estaba confirmado en un commit ANTES de añadirlo a `.gitignore`. ¿Qué ocurre?

- A) Desaparece automáticamente del historial
- B) Sigue estando rastreado; `.gitignore` no afecta a lo que ya estaba en el repositorio
- C) Se borra del disco
- D) Git lo elimina del último commit automáticamente

Pregunta 79

¿Cuándo es mejor crear el `.gitignore` de un proyecto?

- A) Al final, justo antes de terminar el proyecto
- B) Nunca hace falta si trabajas solo en local
- C) Al principio, antes del primer `git add .`
- D) Solo si vas a subir el proyecto a GitHub

Pregunta 80

¿Qué comando usarías para comprobar que un fichero incluido en `.gitignore` ya no aparece como pendiente?

- A) `git config --list`
- B) `git log`
- C) `git init`
- D) `git status`

Bloque 10

```
git diff
```

Pregunta 81

¿Qué muestra `git diff` (sin argumentos)?

- A) Los cambios línea a línea respecto a la última versión confirmada, sin incluir lo ya preparado
- B) El historial completo de commits
- C) La configuración de git
- D) La lista de ficheros ignorados

Pregunta 82

En la salida de `git diff`, ¿qué indica una línea que empieza por `+` ?

- A) Una línea eliminada
- B) Una línea añadida
- C) Un comentario
- D) Un error

Pregunta 83

En la salida de `git diff`, ¿qué indica una línea que empieza por `-`?

- A) Una línea añadida
- B) Una línea sin cambios
- C) Una línea eliminada respecto a la versión anterior
- D) El nombre del autor

Pregunta 84

¿En qué momento del flujo de trabajo tiene más sentido usar `git diff`, antes de confirmar un cambio?

- A) Después de `git commit`
- B) Antes de `git init`
- C) Después de borrar el repositorio
- D) Antes de `git add`, para revisar qué se va a preparar

Pregunta 85

¿ `git diff` modifica los ficheros del proyecto?

- A) No, solo los muestra, es un comando de solo lectura
- B) Sí, aplica los cambios mostrados
- C) Sí, pero solo los ficheros staged
- D) Sí, borra las líneas eliminadas

Pregunta 86

Si no hay ningún cambio sin preparar en el proyecto, ¿qué muestra `git diff` ?

- A) Un error
- B) Nada (no hay diferencias que mostrar)
- C) Todo el contenido de los ficheros
- D) El historial de commits

Bloque 11

Preguntas de escenario

Pregunta 87

Acabas de instalar git por primera vez y ejecutas `git commit -m "primer cambio"` sin haber configurado nada más. ¿Qué es lo más probable que ocurra?

- A) Funciona perfectamente sin problema
- B) Se crea el repositorio automáticamente
- C) Git puede fallar o avisar porque no tiene tu nombre/email configurados
- D) Se sube el proyecto a GitHub

Pregunta 88

Ejecutas `git status` y ves un fichero en "Untracked files". ¿Qué comando necesitas para que en el próximo `git status` aparezca en "Changes to be committed"?

- A) `git commit -m "mensaje"`
- B) `git log`
- C) `git init`
- D) `git add <fichero>`

Pregunta 89

Tienes 3 ficheros modificados: `a.txt`, `b.txt` y `c.txt`. Solo quieres confirmar los cambios de `a.txt` y `b.txt` en este commit. ¿Qué harías?

- A) `git add a.txt b.txt` y luego `git commit -m "..."`
- B) `git add .` y luego `git commit -m "..."`
- C) `git commit -m "..."` `a.txt b.txt` sin usar `git add`
- D) `git init a.txt b.txt`

Pregunta 90

Después del escenario anterior (Pregunta 89), ¿qué mostrará `git status` respecto a `c.txt` ?

- A) Que ya está confirmado
- B) Que sigue con cambios sin preparar (modified)
- C) Que ha desaparecido del proyecto
- D) Que está en `.gitignore`

Pregunta 91

Quieres evitar que un fichero `secrets.env` con contraseñas se suba nunca al historial. Aún no lo has añadido nunca con `git add`. ¿Qué es suficiente?

- A) Borrarlo después de cada commit
- B) Renombrarlo cada vez
- C) Añadirlo a `.gitignore` antes de hacer `git add`.
- D) No hace falta hacer nada especial

Pregunta 92

Mismo caso que la pregunta anterior, pero `secrets.env` ya fue confirmado en un commit anterior por error. ¿Basta con añadirlo ahora a `.gitignore`?

- A) Sí, desaparece automáticamente del historial
- B) Sí, pero solo si se hace `git commit` otra vez
- C) No, hay que borrar la carpeta `.git` entera
- D) No, `.gitignore` no afecta a ficheros que ya estaban rastreados/confirmados

Pregunta 93

Ejecutas `git log --oneline` y ves 3 líneas. ¿Cuántos commits hay en el historial?

- A) 3
- B) 1
- C) 0
- D) Depende de cuántos ficheros haya

Pregunta 94

Quieres saber si tienes bien configurado el email antes de empezar a hacer commits en un proyecto nuevo. ¿Qué comando usarías?

- A) `git log`
- B) `git config --global --list` (o `git config user.email`)
- C) `git status`
- D) `git diff`

Pregunta 95

Has creado una carpeta de proyecto, has añadido ficheros, pero `git status` te dice "not a git repository". ¿Qué te falta?

- A) Configurar el email
- B) Hacer un commit
- C) Ejecutar `git init` en esa carpeta
- D) Instalar Git LFS

Pregunta 96

Modificas un fichero ya confirmado y ejecutas `git add` + `git commit` de nuevo. ¿Se sobrescribe el commit anterior o se crea uno nuevo?

- A) Se sobrescribe el anterior, solo queda 1 commit
- B) Da un error porque el fichero ya estaba confirmado
- C) Git lo ignora porque ya existía
- D) Se crea un nuevo commit, y el historial guarda ambos como snapshots distintos

Pregunta 97

Antes de hacer `git add`, quieres revisar exactamente qué línea has cambiado en `app.py` respecto al último commit. ¿Qué comando usas?

- A) `git diff`
- B) `git log`
- C) `git status`
- D) `git config`

Pregunta 98

Ejecutas por error `git init` dentro de una carpeta que ya era un repositorio git (ya tenía `.git`). ¿Qué es lo más habitual que ocurra?

- A) Se borra todo el historial anterior
- B) Git reinicializa el repositorio existente sin borrar el historial (avisa de que ya existía)
- C) Da un error fatal y detiene el sistema
- D) Se crea un repositorio duplicado dentro de otro automáticamente

Pregunta 99

Tienes un `.gitignore` con `*.log`, pero al ejecutar `git status` sigue apareciendo `errores.log` como untracked. ¿Cuál es la causa más probable?

- A) `.gitignore` nunca funciona con extensiones
- B) `git status` no lee el `.gitignore`
- C) `errores.log` ya estaba siendo rastreado/confirmado antes de añadir la regla al `.gitignore`
- D) Hace falta reiniciar el ordenador

Pregunta 100

Resume el orden correcto del ciclo básico de trabajo en local visto en esta sesión:

- A) `git commit` → `git add` → editar → `git status`
- B) `git init` → `git commit` → `git add` → editar
- C) `git log` → `git init` → `git add`
- D) Editar → `git add` → `git commit` (y vuelta a editar)

¡Suerte con el repaso!